

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-201 - Machine Learning - Fall 2026

Homework 1 - Student Report Worksheet

k-Nearest Neighbors: From Implementation to Honest Evaluation

Name: Emil Jafarov
Submitted: 05/06/2026
Total time: 25 hours

Student ID: [Your ID]
Late days used: 0
Collaborators: None

AI tool usage declaration: I used ChatGPT mainly to understand the usage of numpy broadcasting in this project. Also, AI was used to understand which tests should be written and to review the whole project in different stages.

Executive summary

The best value of *k* turned out to be **1** when analyzed on the validation set. If we analyze the results on the test set, the KNN achieved a **(0.5773)** F1 score. The majority-class baseline has **(0)** F1 score. Also, the other metrics, including precision, recall, and accuracy score, show that the model learned the meaningful patterns related to the survival rate. An interesting and surprising result was observed as the effect of scaling. The best mean cross-validation score increased from **(0.5591)** to **(0.7260)** after scaling. This proves that standardization is specifically quite important for a model like KNN since features with wide ranges can dominate the distance calculation.

1. Exploratory Data Analysis [20 pts]

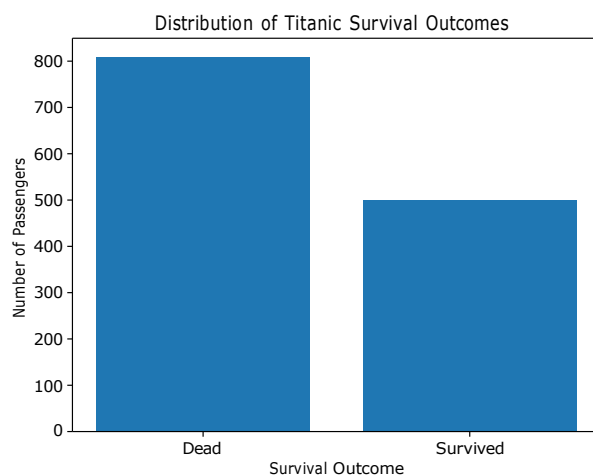
1.1 Summary statistics [4 pts]

	mean	median	std	non-null-count	missing-count
pclass	2.294882	3.0000	0.837836	1309	0
age	29.881138	28.0000	14.413493	1046	263
sibsp	0.498854	0.0000	1.041658	1309	0
parch	0.385027	0.0000	0.865560	1309	0
fare	33.295479	14.4542	51.758668	1308	1

	feature	category	count
0	sex	male	843
1	sex	female	466
2	embarked	S	914
3	embarked	C	270
4	embarked	Q	123
5	embarked	NaN	2

The first data-quality issue was related to the '*age*' column, as the **20.09%** of the values were absent. Also, the '*fare*' feature was right-skewed, since its mean of **33.30** is much higher than the median of **14.45**. This indicates that a small number of passengers paid unusually high fares.

1.2 Target distribution and class balance [4 pts]



The dataset is mildly imbalanced, since **61.80%** of the passengers did not survive while **38.20%** survived, so KNN's majority-vote rule may be somewhat biased toward predicting the non-survival class when neighborhoods contain mixed labels. In the worst case, even (**k** = **1**) could fail to predict the minority class for a test point if its nearest neighbor belongs to the majority

1. Exploratory Data Analysis [continued]

1.3 Three feature-vs-target investigations [4 pts]

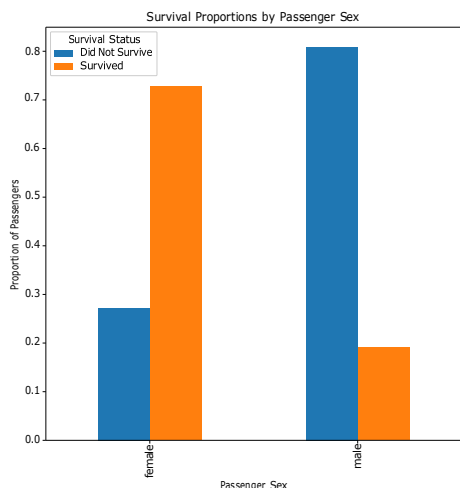


Figure 1: sex vs. survived

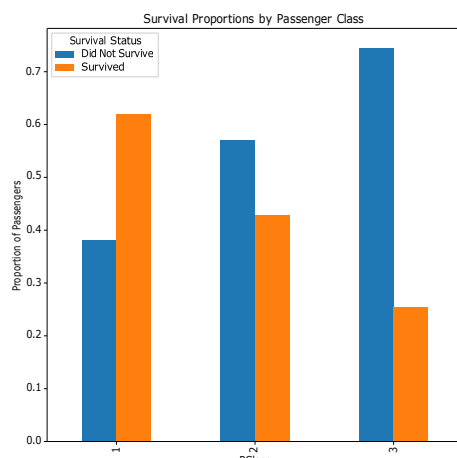


Figure 2: pclass vs. survived

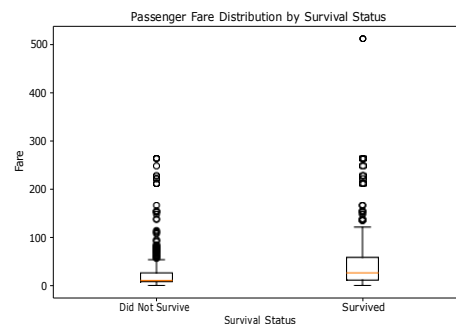


Figure 3: fare vs. survived

The plot comparing the 'sex' feature and the survival data shows a big difference in rates. While **72.75%** of female passengers survived, this number is just **19.10%** for males. The 'pclass' plot also shows a clear pattern, as the survival rate decreases from **61.92%** in first class to **42.96%** in second class and **25.53%** in third class. The 'fare' plot suggests that survivors generally paid higher fares, although the feature contains several high-value outliers. Among these features, the strongest predictor would be 'sex'. If I need another feature to use along with 'sex', it would probably be 'pclass' because it also depicts a general picture for survival rates.

1.4 Missing-data strategy [4 pts]

Feature	Missing	Missing %	Strategy	Your justification and trade-off
age	263	20.09	median imputation	[Write in your own words]
fare	1	0.08	median imputation	[Write in your own words]
embarked	2	0.15	mode imputation	[Write in your own words]

I chose the median method to fill in the missing data for the variables, 'age' and 'fare', respectively, as both variables may be impacted by outliers. 'fare' in particular, is very positively skewed. Therefore, using the median to fill in a missing value is a better alternative since the median is less affected by the outliers and retains the row with the missing value, although filling in a row with a single, central value results in some natural variations from the data being lost. I used mode to fill in the missing data in 'embarked' because the number of missing data values was small, and the most frequently occurring value would be a reasonable substitution, though it could add a slight bias to the observed data. To ensure evaluation of the model would not be impacted through leakage of information from one subset of data to another, any fill values used in the imputation process were calculated solely from the training subset relevant to the model and subsequently applied to the corresponding validation or test subsets of data.

1.5 Encoding and final feature matrix [4 pts]

One-hot encoding is applied for categorical variables ('sex' and 'embarked') because these types of variables do not have any orders associated with their categories. After using one-hot encoding, we can now create our final feature matrix, which has (N = 1,309) rows and (p = 10) columns. This matrix consists of five original features plus two binary columns to indicate the value of 'sex' and three binary columns to indicate the value of 'embarked'. Additionally, because the KNN algorithm computes Euclidean distance based on all of the dimensions in the feature matrix, as the number of dimensions increases, the distance between two points may become less meaningful; therefore, it may be more difficult to identify two examples that are truly close together. In addition, since the binary columns and the continuous numeric columns contribute differently to the calculation of Euclidean distance, scaling of the features is necessary to ensure that the continuous numeric features (such as 'fare') do not dominate the distance calculation.

2. KNN from Scratch [30 pts]

2.1 Class API [8 pts]

```
class KNN:
    def __init__(self, k: int = 5, metric: str = "euclidean",
                 q: float = 2.0, task: str = "classification",
                 weights: str = "uniform") -> None: ...
    def fit(self, X: np.ndarray, y: np.ndarray) -> "KNN": ...
    def predict(self, X: np.ndarray) -> np.ndarray: ...
    def predict_proba(self, X: np.ndarray) -> np.ndarray: ...
```

This implementation provides explicit validation of input and raises errors when inputs are of an invalid type, dimension, if prediction not fitted, if there are not enough rows for training, or feature counts mismatch. I think it is better to create clear errors than to just reshape or coerce inputs without informing the user of it happening as these hidden conversions will create inaccurate predictions.

2.2 Vectorized distance computation [8 pts]

Pairwise-distance computation uses NumPy broadcasting without a Python loop over the training points. Its time complexity is **$O(\text{number of test rows} \times \text{number of training rows} \times \text{number of features})$** . The current implementation creates both a distance matrix and a temporary differences tensor, so memory usage can become large; for 100,000 training rows and 1,000 test rows, the distance matrix alone requires 800,000,000 bytes. Memory usage could be reduced by processing the test rows in smaller batches, although this would require multiple passes.

2.3 Sanity check against sklearn [4 pts]

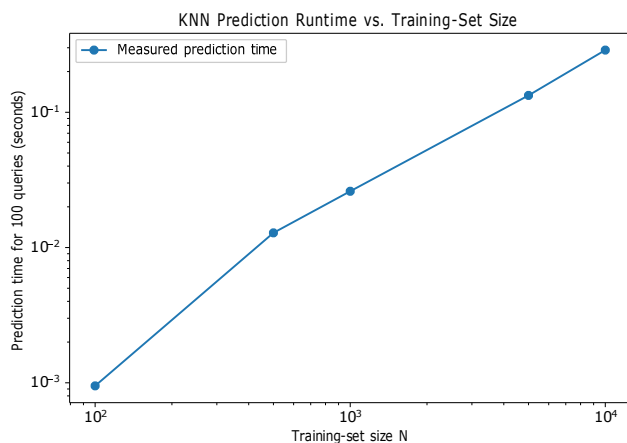
All twenty-one test cases passed. The classifier developed using seed 42 matches sklearn for both predicted match type and predicted probability value; the two are considered equivalent when rounded off to ten (10) decimal points. The logic used to select the nearest neighbors needed to be modified before the resulting model could pass its initial quality assurance tests (sanity checks).

Debugging note: On the Titanic validation set, the custom model and sklearn produced 7 different predictions out of 262 samples because some neighbors had equal distances but different labels. The two implementations can select different neighbors when distances are tied.

2.4 Probability output predict_proba [5 pts]

For binary KNN with uniform voting, there can be 6 distinct probabilities when $k=5$. Same way, 22 probability values exist when $k=21$. Since many samples can receive the same score, ROC thresholds may lead to the same point. That is why we should handle the tied scores.

2.5 Computational benchmark [5 pts]



The measured empirical scaling exponent was approximately **0.984**. This is reasonably close to the expected near-linear growth with training-set size, although the result is slightly affected by NumPy overhead, sorting, memory allocation, and hardware performance. At larger dataset sizes, these additional costs can make prediction time increase somewhat faster than the ideal linear trend.

3. Honest Evaluation [30 pts]

3.1 Stratified split [5 pts]

	Overall	Train	Validation	Test
N	1309	785	262	262
Positive class	38.1971%	38.2166%	38.1679%	38.1679%

Stratified split preserved the balance across all sets. The maximum deviation was **0.029**, which is well below the required 0.5 percentage-point threshold.

3.2 Five metrics from scratch [10 pts]

Accuracy is calculated as the total number of correct predictions divided by the total number of samples in your test data.

Precision calculates how many of the positive predictions made by an algorithm were actually true positives by dividing the number of true positives by the total number of predictions made to be positive. If an algorithm predicts a number of positive predictions, but none actually exist, then it returns a value of 0.

Recall calculates how many of the actual positive samples in the dataset were captured as true positives from the predicted positives by dividing this by the total number of actual positives in the dataset. If an algorithm is run and there are no actual positive samples in the dataset, it returns a value of 0.

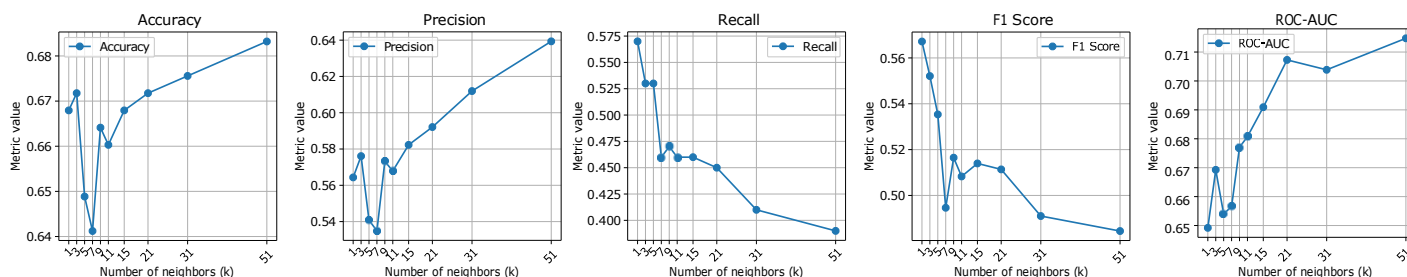
The **F1 score** takes the harmonic mean of precision and recall; therefore, it returns a value of 0 if either precision or recall returns a value of 0. This prevents division by 0.

The **AUC-ROC value** (area under the curve - receiver operating characteristic curve) evaluates how well an algorithm ranks positive samples higher than negative samples across the range of threshold values. The way KNN returns probability scores often results in tied predictions. Thus, when KNN computes the ROC curve, tied predictions are processed as if they belong to the same group rather than having an arbitrary processing algorithm used to determine their location in the ROC curve. This makes the AUC-ROC result consistent with the traditional ROC-AUC implementation found in sklearn.

All metrics produced by the implementation have been tested and match sklearn, including the tied-score ROC-AUC result.

3.3 Hyperparameter sweep over k [8 pts]

Validation Metrics for Different Values of k



Metric	Accuracy	Precision	Recall	F1	ROC-AUC
Best k	51	51	1	1	51
Best value	0.6832	0.6393	0.5700	0.5672	0.7148

"Recall is the measurement that reacted to changes in k in a much more drastic way than the other measurements." This is true because different measurements did not result in similar k values, because they assessed performance differently in that while precision measures prediction reliability, recall concentrates on maximizing the number of people found alive. "I used $k = 1$ based on the validation F1 score." I chose $k = 1$ since the F1 score reflects both precision and recall, and I am dealing with a slight imbalance in binary classification.

3. Honest Evaluation [continued]

3.4 Baselines [4 pts]

Validation model	Accuracy	Precision	Recall	F1	ROC-AUC
Custom KNN ($k = 1$)	0.6679	0.5644	0.5700	0.5672	0.6492
sklearn KNN ($k = 1$)	0.6756	0.5728	0.5900	0.5813	0.6592
Majority baseline	0.6183	0.0000	0.0000	0.0000	0.5000

The modified K-nearest neighbor count model raised the percent of accurate predictions from 0.6183 (majority baseline) to 0.6679 and also raised the F1 metric from 0.0000 to 0.5672. Additionally, it had a precision value of 0.5644 and a recall value of 0.5700, indicating that a significant amount of positive survivor predictions were made rather than just majority votes. The reason for the very small difference in performance between the modified and original implementations is due to the presence of unique distance ties created by duplicate records being encoded exactly the same way, which caused the two algorithms to select different neighbors.

3.5 Final test-set evaluation [3 pts]

Split	Accuracy	Precision	Recall	F1	ROC-AUC
Validation ($k = 1$)	0.6679	0.5644	0.5700	0.5672	0.6492
Held-out test ($k = 1$)	0.6870	0.5957	0.5600	0.5773	0.6627

I used the held-out test set exactly once, after selecting ($k = 1$) based on the validation F1 score. The test results were slightly better overall: F1 increased from 0.5672 to 0.5773, accuracy increased from 0.6679 to 0.6870, and ROC-AUC increased from 0.6492 to 0.6627. The differences are small, so the test performance is consistent with the validation results and can reasonably be explained by normal variation between the two splits.

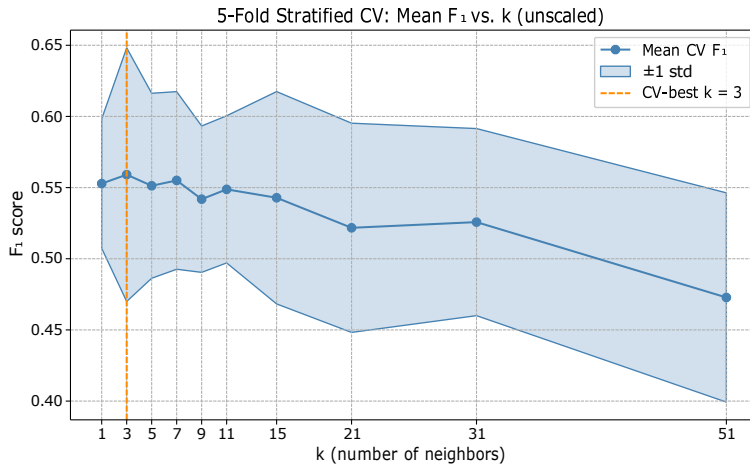
4. Cross-Validation, Bias-Variance, and Scaling [20 pts]

4.1 Stratified K-fold CV [5 pts]

The standard deviation of the positive-class ratios in each validation fold is 0.0998 percent, which comfortably satisfies the requirement of being below 0.5 percent. Additionally, the ratios of each fold are consistent with the results from sklearn's stratified K-fold.

4. Cross-Validation, Bias-Variance, and Scaling [continued]

4.2 CV F1 vs. k [5 pts]

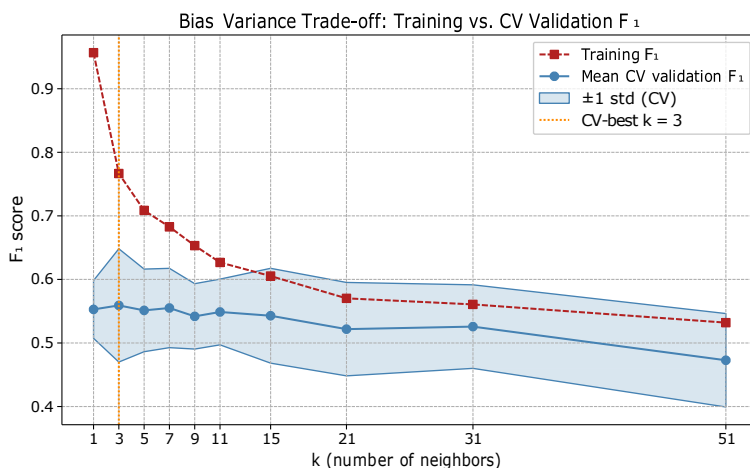


The standard deviation band is relatively wide compared with the small differences in mean F1 near the best values of (k). This means that the single-split choice of ($k = 1$) is less reliable, while the five-fold CV estimate provides a more stable view of performance.

4.3 CV-best k vs. single-split-best k [3 pts]

Single Split Validation used $k = 1$, while Cross Validation used $k = 3$. I find the latter more reliable since it gives an average result over a number of splits, hence being less dependent on randomness during the validation phase. This implies that the former might be a result of sampling error rather than being a better (k) value.

4.4 Bias-variance discussion [4 pts]

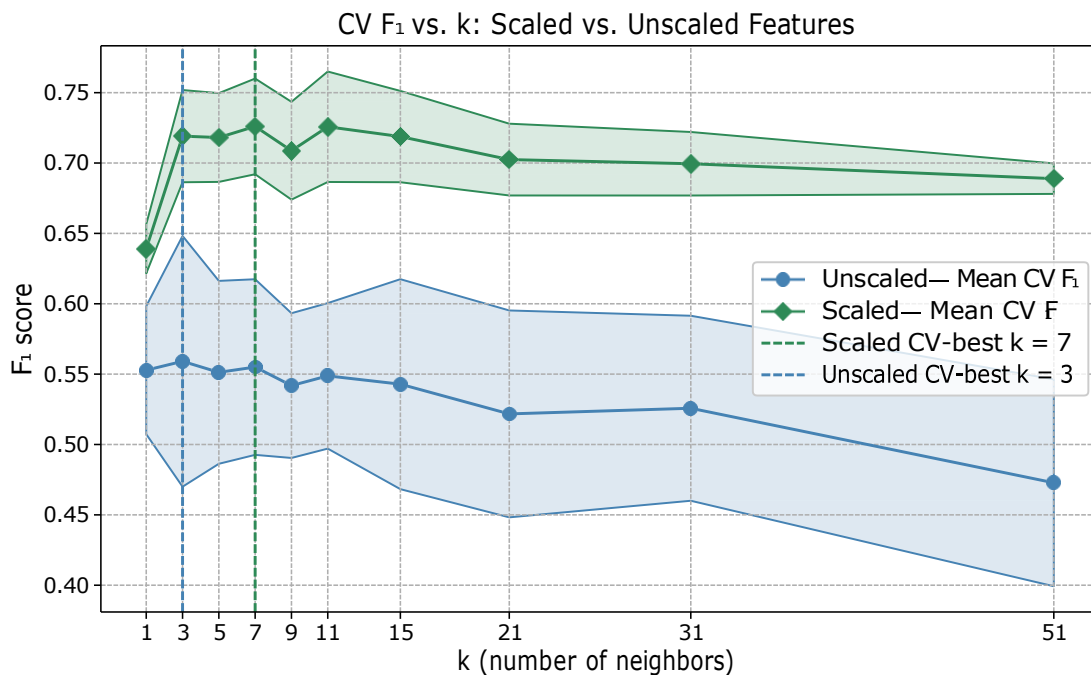


The F1 score for training at ($k = 1$) is 0.9567, whereas the mean F1 score at cross-validation is 0.5528. Such a significant difference means that the model was memorized with respect to the training dataset but cannot generalize well on unseen samples. The increase of (k) to 3 makes the training F1 equal to 0.7666 and the mean F1 in cross-validation higher by 0.0063.

For ($k = 51$), both scores decrease significantly: 0.5320 is achieved in training and 0.4728 in cross-validation. That indicates underfitting since the model becomes too smooth and loses necessary local properties. The optimum found is not a maximum because F1 scores in cross-validation around the optimum do not differ from each other.

4. Cross-Validation, Bias-Variance, and Scaling [continued]

4.5 Scaling experiment [3 pts]



Experiment	Best <i>k</i>	Mean CV F1	Std F1
Unscaled	3	0.5591	0.0891
Scaled, fold-safe	7	0.7260	0.0340
Lift		+16.69 percentage points	

KNN is sensitive to feature scaling because Euclidean distance is strongly affected by the numerical range of each feature. In this dataset, `fare` ranges from 0 to 512.3292, while `age` ranges from about 0.17 to 80, `pclass` ranges only from 1 to 3, `sibsp` from 0 to 8, and `parch` from 0 to 9. Without scaling, differences in `fare` can dominate the distance calculation and make the smaller-range features less influential, even when they contain useful information. To avoid data leakage, the scaler was fitted separately on each training fold and then applied to the corresponding validation fold.

Bonus (optional)

Bonus 1 - Weighted KNN (+5)

I implemented distance-weighted KNN using the weight formula $(1/(d + 10^{-8}))$, so closer neighbors have a greater influence on the prediction. The feature is supported for classification probabilities, classification predictions, and regression predictions. The tests confirmed that closer neighbors receive more influence, weighted regression uses the correct weighted average, and the result matches uniform voting when all selected neighbors are equally distant.

Bonus 2 - Approximate Nearest Neighbors with BallTree (+5)

Not implemented.

Reflection

I would say the effect of feature scaling on performance was the factor that surprised me the most. Understanding how numpy broadcasting must be implemented for KNN, metrics and across the whole project was not simple but quite interesting. This homework helped me understand KNN better and improved my python skills for machine learning.